

Executable BFV Mask Preparation for Public-Weight Language-Model Inference: A Historical PLLM Study

Blair Hudson

11 September 2026

Abstract

This paper documents a historical, executable study of additive masking for public-weight integer linear layers. A client prepares fresh pairs (r, Wr) by encrypting masks with BFV; online evaluation then uses ordinary masked modular matrix multiplication. The artifact completes a coefficient-matrix evaluator through SEAL ciphertext reconstruction and client decryption. On one retained CPU environment, the median complete preparation time for a matched 768 by 256 stage falls from 3.122 to 0.633 seconds over three samples, a 4.93x speedup. Five Qwen3.5-27B-shaped stages are executed with synthetic signed four-bit matrices and 2,048 masks per stage. A separate small trained decoder exercises preparation, depletion, masked execution, and decoding across client and server processes; each recorded configuration is one run of 96 generated token positions. Capacity calculations derived from the public model configuration project a 48.22 GiB raw inventory for 2,048 complete token sets and a 1.34 token/s aggregate-link ceiling at 1 Gbit/s under idealized compact transfer. These are projections, not achieved model rates. No Qwen checkpoint, GPU, physical wide-area network, language-quality evaluation, formal security proof, or current PLLM runtime benchmark is reported.

1 Introduction

Private inference papers can make an online matrix kernel look inexpensive while leaving preparation, client cryptography, transfer, startup, and discarded work outside the reported rate. Autoregressive decoding makes that omission important: one output token traverses many serial stages, and a prepared mask pair can be used only once.

This paper examines one narrow design point. Weights are public signed integers. A trusted client keeps activations and model state local, while a remote service applies the public matrices. Before inference, BFV encryption prepares additive linear correlations. During inference, no ciphertext operation is required for a linear stage; the service evaluates ordinary modular matrix multiplication on a masked activation. The construction is related to input-independent correlated preprocessing (Beaver 1992), but this study neither introduces a new cryptosystem nor claims a general secure-

computation protocol.

The work is historical in a precise sense. All numerical results come from `research/lifecycle`, whose project version is `0.14.0+inference.study1`. That artifact was derived from the persisted PLLM 0.14 research baseline. The repository’s current mixed Python/Rust PLLM runtime is a separate implementation. It is not exercised here, and no timing or security conclusion in this paper transfers to it without new validation.

The study contributes four reproducible observations. First, it completes an integer coefficient path through valid BFV ciphertext reconstruction and decryption. Second, it measures complete preparation for five dense stage shapes taken from a public large-model configuration, while using synthetic weights. Third, it runs a small learned hybrid decoder across isolated processes from an empty mask inventory. Fourth, it separates those measurements from model-scale capacity calculations. This separation is the central reporting result: exact arithmetic at a target shape is not full-checkpoint inference, and a derived ceiling is not measured throughput.

2 System and Threat Model

2.1 Parties and data

For each linear stage, the client holds an activation vector x , a BFV secret key, fresh masks, activation scales, and local recurrent or attention state. The service holds a public integer matrix W and public arithmetic parameters. Model identity, matrix shapes, request counts, timing, batch occupancy, and preparation schedules are not hidden. The embedding is local in the model-scale plan.

The intended confidentiality goal is limited to inputs against a service that follows the prescribed computation while inspecting its transcript. Weights are explicitly public. The study does not address model confidentiality, malicious-client queries, malicious-service correctness, traffic-analysis resistance, denial of service, or collusion with the trusted client environment.

For a fresh uniform r , the online value $x + r \pmod{p}$ is uniform for fixed x . That algebraic fact does not by itself prove security of the combined preparation and online proto-

col. The intended argument also depends on BFV hiding r , correct fresh randomness, one-time correlation use, and the stated service behavior. The BFV construction is described in its original paper (Fan and Vercauteren 2012); the implementation uses Microsoft SEAL through TenSEAL (Microsoft Research 2026; Benaissa et al. 2021).

2.2 What the prototype checks

The prototype uses operating-system randomness with rejection sampling for masks. Framed messages use HMAC-SHA256 plus sequence and epoch fields; retained tests reject modified authentication data, repeated frames, repeated correlation identifiers, and old epochs. These tests establish behavior of the tested implementation, not verifiable execution of W or a malicious-security theorem.

SEAL accepts the principal parameter set when asked to enforce its `tc128` level. The Homomorphic Encryption Standard gives broader parameter guidance (Albrecht et al. 2018), but this paper does not independently estimate security. Measured invariant-noise budgets only show margin for the tested ciphertext operations. They are not a security level or a substitute for cryptographic review.

3 Protocol

3.1 Prepared linear masking

Let $W \in \mathbb{Z}^{m \times n}$ and let arithmetic use $\mathbb{Z}_p$. During preparation, the client samples $r \leftarrow \mathbb{Z}_p^n$, sends $\text{Enc}(r)$, and retains r . The service returns $\text{Enc}(Wr)$, which the client decrypts and retains. During online evaluation, the client sends $d = x + r \pmod{p}$, the service returns $u = Wd \pmod{p}$, and the client computes

$$u - Wr = W(x + r) - Wr = Wx \pmod{p}. \quad (1)$$

Reusing r reveals the modular difference between two inputs because $(x_1 + r) - (x_2 + r) = x_1 - x_2$. Consequently, submission burns the correlation even after cancellation, failure, or proposal rejection. Reusing already recovered values inside local deterministic computation is different and does not resend a masked input.

3.2 Coordinate packing

For $B \leq N$ future masks, coordinate i is encoded as

$$P_i(X) = \sum_{b=0}^{B-1} r_{b,i} X^b. \quad (2)$$

The service evaluates $Q_j = \sum_i W_{j,i} \text{Enc}(P_i)$. Coefficient b of the decrypted Q_j is $(Wr_b)_j \pmod{p}$. Scalar linear combinations do not mix polynomial coefficients, so this evalua-

tor uses no rotations, Galois keys, bootstrapping, relinearization, or ciphertext-ciphertext multiplication. Masks packed in one ciphertext share one client key; this layout does not combine unrelated customers.

In the measured profile, a BFV ciphertext has two polynomials, one coefficient prime q , and $N = 2,048$ coefficients per polynomial. Representing n ciphertexts as an n by $2N$ residue matrix A gives

$$C = WA \pmod{q}. \quad (3)$$

The server splits residues into eight-bit digits, shifts each digit into signed `int8`, evaluates exact integer GEMM with PyTorch (Paszke et al. 2019), restores the public offset with row sums, and recombines digits modulo q . The implementation checks `int32` dot-product bounds and uses `int64` Horner accumulation. It does not use floating-point GEMM for ciphertext coefficients.

3.3 Library boundary and signed range

The bridge parses the measured SEAL 4.3.3 serialization layout and reconstructs ciphertexts accepted by SEAL’s loader. It accepts only coefficient-form BFV ciphertexts with two polynomials and one prime. This is an artifact-specific bridge, not a general SEAL serialization API.

Ring exactness is not enough to recover an intended signed dot product. For W4A4 values in $[-7, 7]$, the magnitude is at most $49n$; centered recovery therefore requires $p > 98n$. The principal $p = 2,097,169$ covers the largest tested width, 17,408. This bound says nothing about quantization error relative to a floating checkpoint.

4 Artifact and Targets

4.1 Historical artifact

The executable study is under `research/lifecycle`. Its Python project pins Python 3.13, NumPy 2.3.5, PyTorch 2.10.0, and TenSEAL 0.3.17 through `pyproject.toml` and `uv.lock`. TenSEAL 0.3.17 vendors SEAL 4.3.3. The coefficient bridge, preparation evaluator, online masked evaluator, planner, small decoder, recurrent benchmark, and 45-test historical suite are all in that directory. Retained JSON records are mirrored under `research/evidence`.

`paper/IMPLEMENTATION-STATUS.md` records the implementation boundary. In particular, current PLLM package tests, current Rust kernels, current serving behavior, and current security properties are outside this paper. The word “artifact” below always means the historical lifecycle study.

4.2 Large target shapes

The planner reads dimensions from the public Qwen3.5-27B configuration and the corresponding Transformers implemen-

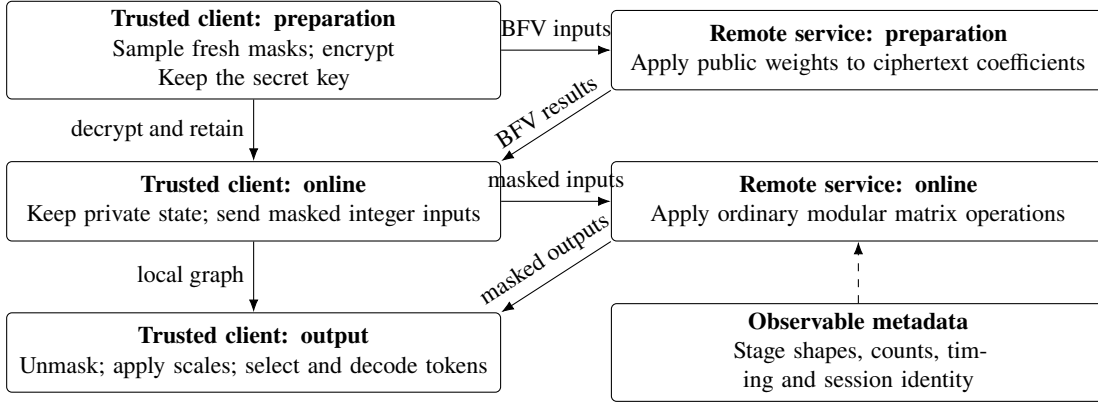


Figure 1: Studied execution boundary. Preparation belongs in the resource budget. The client is trusted; the service-side online operation is masked modular arithmetic.

Stage	Input	Output	Count
DeltaNet input	5,120	16,480	48
Attention input	5,120	14,336	16
Mixer output	6,144	5,120	64
MLP expansion	5,120	34,816	64
MLP contraction	17,408	5,120	64
Vocabulary head	5,120	248,320	1

Table 1: Config-derived public text plan. It contains 257 serial dense-stage calls and 25.62 billion matrix elements per model copy. No full checkpoint was loaded.

tation (Qwen Team 2026; Wolf et al. 2020). The configuration describes 64 text blocks: 48 Gated DeltaNet blocks and 16 gated-attention blocks. Gated DeltaNet supplies the recurrent structure (Yang et al. 2025). Projections are fused only when they consume the same activation.

Every target-shaped benchmark matrix is deterministic synthetic W4 data. Dimensions and invocation counts are config-derived; weights, timings, and arithmetic checks do not establish Qwen model quality. The local-embedding assumption also has a cost: the plan derives 606.25 MiB for ideal int4 storage or 2.37 GiB at BF16, before other client state.

4.3 Lifecycle fixture

The separate lifecycle target is a trained four-block, width-32 decoder: three DeltaNet blocks, one gated-attention block, causal convolution, partial rotary embeddings, RMS normalization, SwiGLU, and a vocabulary head. It trains for 160 steps on a short authored corpus. This learned fixture exercises control flow and exact W4A4 agreement; it is not a language-quality dataset or a scaled model surrogate.

5 Evaluation Method

5.1 Evidence classes

We use three labels throughout. **Measured** values are retained wall-clock or correctness observations from executed artifact code. **Config-derived** values are arithmetic over saved public dimensions and counts. **Projected** values combine measurements with config-derived counts or idealized bandwidth assumptions. Synthetic target-shaped execution is measured, but it remains synthetic rather than checkpoint execution.

The retained benchmark host ran Linux 6.18.35 on an AMD EPYC 9V74 environment exposing five logical CPUs, a quota of four CPU equivalents, and 4 GiB of memory. It used Python 3.13.5, NumPy 2.3.5, CPU-only PyTorch 2.10.0, and TenSEAL 0.3.17. No GPU or physical network was used.

5.2 Samples and timing boundaries

The matched 768 by 256 control and coefficient paths each retain three complete samples. Each of the five target-shaped preparation stages retains two samples. The 1,024-row head preparation tile retains two samples. Complete preparation timing includes mask sampling, encryption/export, coefficient conversion, server evaluation, ciphertext import, and decryption. It excludes key setup, correctness-reference calculation, and physical transfer. Raw first-use observations remain in the JSON.

Compiled online matrix benchmarks retain four timings for each non-head stage at batches 1, 8, and 16. The separately allocated full 5,120 by 248,320 head retains three timings per batch. The compact serialization comparison retains three samples. The W4A8 parameter check retains one execution. Recurrent timing groups retain seven samples.

Each cell in the lifecycle latency table is one complete run, not a median or repeated trial. There are six configurations: ordinary and four-proposal decoding at injected delays of 0, 1, and 10 ms. Each run trains the same deterministic fixture

Matrix stage	Batch time (s)	Correlations/s
768 to 256, control	3.122	656.1
768 to 256, coefficient	0.633	3,235.3
5,120 to 16,480	16.56	123.7
5,120 to 14,336	15.24	134.4
6,144 to 5,120	8.39	244.2
5,120 to 34,816	32.26	63.5
17,408 to 5,120	19.03	107.6

Table 2: Measured actual BFV preparation with 2,048 masks per stage. Matched rows have three samples each; target-shaped rows have two. Target-shaped weights are synthetic W4.

and generates 96 token positions after an 18-token prompt. Small sample counts preclude confidence intervals and broad hardware conclusions.

5.3 Correctness checks

Target-sized preparation checks compare every output column for 16 mask rows with independent NumPy integer arithmetic. Small BFV tests compare every row. Coefficient tests compare against unbounded Python arithmetic and SEAL ciphertext additions. Full-head online checks compare 128 output rows for every batch row. The historical JUnit record reports 45 tests, zero failures, zero errors, and zero skips.

6 Results

6.1 Complete BFV preparation

Table 2 reports medians. At 768 by 256, the matched control median is 3.122 s and coefficient-GEMM preparation is 0.633 s, a 4.93x speedup. This is a complete preparation comparison for 2,048 correlations, not a generated-token rate.

For the expansion, median server GEMM is 15.46 s, client encryption/export 2.35 s, and client import/decryption 13.82 s. For the contraction, client encryption/export alone is 8.80 s. These component measurements show that speeding only the server coefficient GEMM does not remove client cost.

SEAL’s seeded symmetric serialization plus a lossless seven-byte coefficient encoding reduces the measured 768 by 256 payload from 33.55 MB to 19.31 MB, 42.5%. The median local encode/evaluate/decode path is 0.691 s versus 0.633 s for the raw coefficient path. This is a transfer-size tradeoff; no network was measured.

6.2 Online arithmetic and activation range

The synthetic full vocabulary matrix is allocated and executed online. Median masked times for batches 1, 8, and 16 are 0.115, 0.233, and 0.357 s per batch, respectively, from three samples each. Preparation for that full head is not mea-

Added delay	Ordinary (s)	Proposals (s)	Ratio
0 ms	2.90	4.27	0.68
1 ms	4.91	5.35	0.92
10 ms	20.11	11.29	1.78

Table 3: Measured single complete run per cell, each generating 96 token positions. Delay is inserted once per online exchange. Ratio is ordinary/proposals.

sured: only a 1,024-output preparation tile is measured and later projected.

A separate one-run arithmetic check raises activations from W4A4 to W4A8. At width 17,408, $p = 33,554,467$ recovers signed extrema $\pm 15,475,712$ exactly and leaves a measured 14-bit invariant-noise budget. Online residues require four bytes instead of three. This is parameter compatibility for synthetic extrema, not evidence of checkpoint accuracy.

6.3 Small lifecycle execution

The ordinary zero-delay run starts with an empty inventory, prepares 2,176 correlations, consumes 1,904, and leaves 272. It takes 2.899 s, of which 2.012 s is recorded preparation; clear W4A4 execution takes 0.240 s. First masked token latency is 1.090 s. A second preparation cycle is visible after initial inventory depletion.

Prompt-lookup proposals evaluate four candidates per verification block. At zero delay, calls fall from 1,632 to 646 (60.4%), while consumed correlations rise from 1,904 to 3,434. Rejected rows still consume fresh material. Table 3 therefore reports both favorable and unfavorable cases.

For all six runs, the retained `exact_tokens` field is true relative to the corresponding clear W4A4 execution. Exact final-logit equality is retained only for the three ordinary runs; proposal records leave that field unset. The repeated fixture favors prompt lookup, and the inserted application delay is not a wide-area-network measurement. The artifact does not implement the general distribution-preserving speculative sampler described by Leviathan et al. (2023).

6.4 Recurrent replay

The recurrent benchmark uses synthetic activations for one Qwen-sized layer. Across 48 DeltaNet layers, eight complete FP32 snapshots derive to 1.125 GiB. Saved projection traces derive to 27.14 MiB plus the 144 MiB base state, about 42x less incremental storage. A four-position same-kernel replay has a 1.962 ms median over seven samples and exactly equals the saved state from that same kernel.

Changing from the elementwise reference to the matrix-kernel implementation changes floating summation order. The retained maximum absolute errors are 1.49×10^{-8} for output and 5.96×10^{-8} for final state, not bit equality.

Batch	Clear s/row	Masked s/row	Masked rows/s
1	0.710	1.976	0.506
8	0.217	0.731	1.369
16	0.101	0.395	2.534

Table 4: Projected dense work for the public 27B text plan, using measured synthetic stage medians. These are not full-model rates.

These are synthetic FP32 observations from one layer, not checkpoint-parity evidence.

7 Capacity Analysis

7.1 Dense work projection

Summing measured per-stage online medians using the config-derived counts in Table 1 produces Table 4. It is a projection of dense integer work. It excludes preparation, transfer, attention, recurrence, normalization, embedding access, and all other graph work. Rows in a batch can represent concurrent sessions or candidate positions; aggregate row rate is not one user’s generation rate.

The same method projects 2.486 s of summed serial stage wall time per complete token set. Full-head output work is scaled from the measured 1,024-row tile to 248,320 rows while common input work is counted once. Preparing 2,048 complete sets projects to 5,091 s (84.9 min) on the benchmark host. No complete model preparation run was performed.

7.2 Communication, inventory, and interaction

The plan contains 2,167,808 input and 4,152,320 output values per decode step. Three-byte online residues derive to 18.96 MB per token. Raw coordinate preparation adds 101.12 MB per full-occupancy token set. Applying the measured seeded-input and seven-byte-output formats projects preparation to 74.63 MB.

On an ideal aggregate 1 Gbit/s link shared by both directions, compact preparation plus online payload has a 1.34 token/s ceiling. If each direction independently provides 1 Gbit/s, the corresponding ceiling is 1.77 token/s. Both ignore headers, compute, contention, startup, and unused preparation. The 6.59 token/s online-only value omits the larger preparation stream and is not a sustained bound.

Storing uint32 input and output masks for 2,048 complete sets derives to 48.22 GiB. Regenerating input masks from a securely managed seed would still leave 31.68 GiB of output masks. These figures are storage calculations, not measured resident memory. At 257 serial stage exchanges, 20 ms of delay per exchange alone derives to 5.14 s per token before arithmetic or transfer.

8 Related Work

BFV provides the exact encrypted arithmetic used during preparation (Fan and Vercauteren 2012). SEAL and TenSEAL provide the implementation boundary (Microsoft Research 2026; Benaissa et al. 2021), while the HE Standard provides parameter-selection context (Albrecht et al. 2018). This study’s prepared (r, Wr) pairs share the broad offline/online motivation of correlated preprocessing such as Beaver’s circuit randomization (Beaver 1992), but they are specialized to public linear maps and generated here with BFV.

Cheetah designs two-party neural-network linear protocols that avoid expensive rotations and combines them with protocols for nonlinear operations (Huang et al. 2022). THE-X and Iron study private Transformer inference under different model, approximation, and protocol choices (Chen et al. 2022; Hao et al. 2022). In contrast, this artifact leaves nonlinear and stateful operators at the trusted client, makes weights public, and does not evaluate a complete pretrained Transformer. It therefore should not be compared as an end-to-end replacement.

Integer-only inference and post-training language-model quantization require accuracy evaluation against floating models (Jacob et al. 2018; Frantar et al. 2023). The present W4 and W4A8 experiments check modular range and exactness only. PyTorch supplies integer kernels (Paszke et al. 2019), and Transformers plus the Qwen model metadata supply target shapes (Wolf et al. 2020; Qwen Team 2026); neither use implies checkpoint execution.

Gated DeltaNet defines the recurrent update used by the target plan (Yang et al. 2025). TreeWY, by Sneha Murthy Ghantasala, develops tree-structured WY verification and accepted-state reconstruction for Gated DeltaNet hybrids (Ghantasala 2026). The historical artifact instead performs simple sequential replay of an accepted prefix. It does not claim TreeWY’s method or results.

9 Limitations and Operational Risk

No large checkpoint was downloaded or loaded. Target-shaped matrices are synthetic; model dimensions and counts come from saved configuration analysis. The study reports no language quality, long-context behavior, GPU result, physical network result, multi-user scheduler, energy use, or full-model throughput. Two or three timing samples cannot characterize variance, and lifecycle table cells have only one run.

The threat model is semi-honest and informal. There is no simulation-based proof, independent parameter review, ciphertext-validation analysis for hostile responses, verifiable model execution, malicious-client policy, or protection for proprietary weights. Transport authentication does not establish correct matrix evaluation. Invariant-noise measurements are correctness diagnostics under the tested path, not evidence against active attacks.

Operational correctness depends on never reusing a correlation. Both processes mark material consumed before replying and reject old epochs in the demonstration, but restoration of a whole process or virtual-machine snapshot is not handled. Cancellation and ambiguous network failure must burn reserved material. Persistent recovery would require a freshness source outside any restored state.

The bridge depends on an inspected SEAL serialization representation and should not parse untrusted arbitrary ciphertexts without a separate review. Mutable target source URLs are recorded with the evidence, but an immutable external Qwen/Transformers revision was not captured; the saved `model-plan.json` is therefore the authoritative target-shape record. Current PLLM runtime code has changed since this artifact and requires its own protocol, correctness, and performance evaluation.

10 Reproducibility

The canonical paper source is `paper/manuscript.md`; citations are maintained in `paper/references.bib`. `scripts/build_paper.py` generates `paper/main.tex`, compiles `paper/main.pdf`, writes content hashes and tool versions to `paper/build-metadata.json`, and creates `paper/arxiv-source.tar.gz`. The archive contains a root `main.tex` whose references are already rendered, plus a build note and machine-readable metadata. The canonical Markdown, bibliography, and Lua filter remain in the repository and the separate website source download. Website article generation remains the separate web target.

From repository root, reproduce the retained historical test suite with:

```
cd research/lifecycle
uv sync --frozen --group test
uv run --frozen python -m pytest -q
```

The retained JUnit file `study-tests.xml` records 45 passed tests. This count belongs to the historical artifact, not the current repository-wide suite. Zstandard must be available to the serialization bridge; on macOS the recorded setup used Homebrew’s `zstd`.

Representative experiment invocations, run from `research/lifecycle`, are:

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=4 uv run
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 uv run
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 uv run
```

Full target runs require substantially more time and memory; `run-optimized.sh` records their original order. Retained claims should be checked against `research/evidence/CLAIMS.json`, then against the named raw JSON rather than regenerated timing on different hardware.

Build paper and arXiv files from repository root with:

```
uv run --no-project --python 3.13 python scripts/build_paper.py
```

Build the historical fixture itself with its locked environment:

```
cd research/lifecycle
uv build
```

The build requires Pandoc and Tectonic or pdfLaTeX. `SOURCE_DATE_EPOCH` can pin generated timestamps; otherwise the script uses the repository commit time and records it. arXiv category, submission license, final author list and affiliations, and DOI policy are submission metadata, not inferred by the build.

11 Conclusion

The historical artifact demonstrates an executable BFV path for preparing one-time additive masks for public integer matrices. Its strongest measured comparison is narrow: at 768 by 256 and batch 2,048, complete median preparation changes from 3.122 to 0.633 s on one retained CPU environment. Target-shaped synthetic runs show that ciphertext-coefficient GEMM can complete large stages, while client cryptography, preparation traffic, inventory, and serial interaction remain material costs.

The small decoder confirms that the artifact can connect preparation, depletion, masked linear calls, local state, and token selection across processes. It does not validate a Qwen checkpoint or current PLLM. Keeping measured, config-derived, projected, and unevaluated claims separate is therefore not presentation detail; it defines the result.

References

- Albrecht, Martin, Melissa Chase, Hao Chen, et al. 2018. *Homomorphic Encryption Security Standard*. HomomorphicEncryption.org. <https://homomorphicencryption.org/standard/>.
- Beaver, Donald. 1992. “Efficient Multiparty Protocols Using Circuit Randomization.” *Advances in Cryptology – CRYPTO ’91*, 420–32. https://doi.org/10.1007/3-540-46766-1_34.
- Benaissa, Ayoub, Bilal Rattat, Bogdan Chelariu, and Alaa Eddine Belkhal. 2021. *SEAL: A fast library for encrypted tensor operations using Homomorphic Encryption*. arXiv:2104.03152. <https://arxiv.org/abs/2104.03152>.
- Chen, Tianyu, Hangbo Bao, Shaohan Huang, et al. 2022. “THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption.” *Findings of the Association for Computational Linguistics: ACL 2022*, 3510–20. <https://doi.org/10.18653/v1/2022.findings-acl.277>.

- Fan, Junfeng, and Frederik Vercauteren. 2012. *Somewhat Practical Fully Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2012/144. <https://eprint.iacr.org/2012/144>.
- Frantar, Elias, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers.” *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.17323>.
- Ghantasala, Sneha Murthy. 2026. *TreeWY: Speculative Verification for Gated DeltaNet Hybrids*. arXiv:2608.20961v1. <https://arxiv.org/abs/2608.20961v1>.
- Hao, Meng, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. “Iron: Private Inference on Transformers.” *Advances in Neural Information Processing Systems* 35, 15718–31. <https://doi.org/10.52202/068431-1143>.
- Huang, Zhicong, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. “Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference.” *31st USENIX Security Symposium*, 809–26. <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong>.
- Jacob, Benoit, Skirmantas Kligys, Bo Chen, et al. 2018. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2704–13. https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html.
- Leviathan, Yaniv, Matan Kalman, and Yossi Matias. 2023. “Fast Inference from Transformers via Speculative Decoding.” *Proceedings of the 40th International Conference on Machine Learning*. <https://arxiv.org/abs/2211.17192>.
- Microsoft Research. 2026. *Microsoft SEAL*. V. 4.3.3. Released. <https://github.com/microsoft/SEAL/tree/v4.3.3>.
- Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Advances in Neural Information Processing Systems* 32. <https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- Qwen Team. 2026. *Qwen3.5-27B Model Card and Configuration*. <https://huggingface.co/Qwen/Qwen3.5-27B>.
- Wolf, Thomas, Lysandre Debut, Victor Sanh, et al. 2020. “Transformers: State-of-the-Art Natural Language Processing.” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>.
- Yang, Songlin, Jan Kautz, and Ali Hatamizadeh. 2025. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. *International Conference on Learning Representations*. <https://arxiv.org/abs/2412.06464>.